

Práctica. CWE-401: Missing Release of Memory after Effective Lifetime.

Instrucciones. Elaborar una aplicación de consola en C que permita ejemplificar la vulnerabilidad de Falta la liberación de memoria después de la vida útil efectiva que permita saltarse una instrucción en equipos Linux.

1. Sigue las instrucciones de la práctica en la siguiente página.
2. Crea un reporte de práctica en un **archivo de Word** con una portada con tu nombre. Guárdalo con el nombre **NombreAlumno_CWE-401.docx**.
3. Coloca **las siguientes capturas de pantalla**, cada una con una **descripción textual** arriba de la captura.
 - a. Coloca la captura de pantalla de tu código fuente de C en Visual Studio Code.
 - b. Coloca la captura de pantalla de la salida del programa **Valgrind** analizando la memoria con el siguiente comando:

```
$ valgrind -s --tool=memcheck --leak-check=full ./programa.out
```

- c. Coloca la captura de pantalla de tu código fuente de C en Visual Studio Code corregido.
 - d. Coloca la captura de pantalla de la salida del programa **Valgrind** analizando la memoria con el siguiente comando
 - e. Publica tu código fuente en un proyecto público de GitHub. **Coloca la URL** del código fuente publicado en GitHub.
4. Sube tu reporte en un archivo Word a la actividad en Eminus.

Prerrequisitos

1. Esta práctica tiene como prerrequisito haber realizado la práctica de instalación de servidor **Linux**.
Puede seguir las instrucciones de cómo instalar Ubuntu Server desde la práctica [Instalación de Ubuntu server](#).
2. Esta práctica tiene como prerrequisito tener instalado el compilador para C/C++ en su equipo.
Puede seguir las instrucciones de cómo instalar el compilador para C/C++ desde la práctica [Instalación del compilador para C/C++](#).
3. Esta práctica tiene como prerrequisito tener instalado el software **Visual Studio Code** con la extensión para C++.
Puede seguir las instrucciones de cómo instalar Visual Studio Code desde la práctica [Instalación de Visual Studio Code](#).
4. Puede seguir las instrucciones de cómo publicar un proyecto en Visual Studio Code a GitHub desde la práctica [Como publicar un proyecto a GitHub desde Visual Studio Code](#).

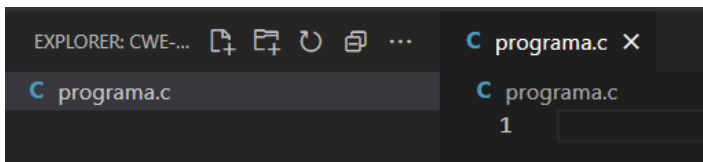
Instrucciones

Esta vulnerabilidad consiste en que por medio de una programación incorrecta, el programa no realiza un seguimiento suficiente ni libera la memoria asignada después de su uso, lo que consume lentamente la memoria restante.

La mayoría de las pérdidas de memoria resultan en problemas generales de confiabilidad del software, pero si un atacante puede desencadenar intencionalmente una pérdida de memoria, el atacante podría lanzar un ataque de denegación de servicio (al bloquear el programa) o aprovechar otro comportamiento inesperado del programa resultante de una falla de condición de memoria baja.

A. Crear el programa

1. Asegúrese de contar con una carpeta de trabajo en **C:\codigo**. Si aún no ha creado la carpeta **C:\codigo** hágalo antes de continuar y compártala con su equipo **Linux**. Puede seguir las instrucciones de cómo hacerlo desde [Compartir su carpeta de trabajo de Windows hacia Linux](#).
2. Inicie **Visual Studio Code**.
3. Seleccione **Archivo > Abrir carpeta** en el menú principal.
4. En el cuadro de diálogo **Abrir carpeta**, cree una carpeta en la ruta **C:\codigo\ps\cwe-401** y selecciónela. Luego haga clic en Seleccionar carpeta.
5. El nombre de la carpeta se convierte en el nombre del proyecto y el nombre del espacio de nombres de forma predeterminada.
6. En la barra de herramientas del Explorador de archivos, seleccione el botón **Nuevo archivo**.
7. Asigne un nombre al archivo **programa.c** y VS Code lo abrirá automáticamente en el editor.



8. Escriba el siguiente código.

```
#include <stdlib.h>
#include <stdio.h>

#define LOOPS 100
#define MAXSIZE 256

int main(int argc, char **argv)
{
    int count = 0;
    char *pointer = NULL;

    for (count = 0; count < LOOPS; count++)
    {
        pointer = (char *)malloc(sizeof(char) * MAXSIZE);

        free(pointer);
    }

    return count;
}
```

9. En este ejemplo, tenemos 10 asignaciones de tamaño `MAXSIZE`. Todas las asignaciones, excepto la última, se pierden. Si ningún puntero apunta al bloque asignado, es irrecuperable durante la ejecución del programa. Una solución sencilla a este ejemplo trivial es colocar la `free()` llamada dentro del bucle `for`.
10. Vamos ahora a compilarlo en su equipo **Linux**.
11. Abra una nueva **Terminal**, colóquese en la ubicación de la carpeta compartida de este código. En este ejemplo es `cd codigo/ps/cwe-401/`.
12. Escriba el siguiente comando para compilar su archivo.

```
$ gcc programa.c -o programa.out -fno-stack-protector -z execstack -ggdb
```

13. Ejecute su programa.

```
pato@equipoA:~/codigo/ps/cwe-401$ ./programa.out
pato@equipoA:~/codigo/ps/cwe-401$
```

14. Observe que no sucede nada más que el uso incorrecto de una pequeña porción de memoria. Pero que pasaría si el ciclo fuera infinito por error de programación, podríamos tener un problema más grave.
15. Evitar pérdidas de memoria en las aplicaciones es difícil incluso para los desarrolladores más expertos. Afortunadamente, existen herramientas que ayudan a rastrear dichas pérdidas de memoria.
16. Un ejemplo de ello en el entorno Unix/Linux es **Valgrind** que ejecuta el programa deseado en un entorno tal que se verifican todas las rutinas de asignación y desasignación de memoria. Al final de la ejecución del programa, **Valgrind** mostrará los resultados de una manera fácil de leer.
17. Instale la herramienta **Valgrind**.

```
sudo apt-get install valgrind
```

18. Si la herramienta no se pudiera instalar usando `apt`, se puede hacer de manera manual usando los fuentes. Colocate en tu carpeta `home`.

```
cd
wget https://sourceware.org/pub/valgrind/valgrind-3.23.0.tar.bz2
tar xvf valgrind-3.23.0.tar.bz2
cd valgrind-3.23.0/
./configure
make
sudo make install
sudo nano ~/.bashrc
# Agrega en la ultima linea del archivo
export VALGRIND_LIB=/usr/local/libexec/valgrind
# Sal del archivo
source ~/.profile
```

19. Ahora analicemos nuestro programa con esta herramienta.

```
$ valgrind -s --tool=memcheck --leak-check=full ./programa.out
```

20. Observe la salida.

```
pato@equipoA:~/codigo/ps/cwe-401$ valgrind -s --leak-check=full ./programa.out
==1269== Memcheck, a memory error detector
==1269== Copyright (C) 2002-2017, and GNU GPL'd, by Julian Seward et al.
==1269== Using Valgrind-3.18.1 and LibVEX; rerun with -h for copyright info
==1269== Command: ./programa.out
==1269==
==1269==
==1269== HEAP SUMMARY:
==1269==   in use at exit: 25,344 bytes in 99 blocks
==1269==   total heap usage: 100 allocs, 1 frees, 25,600 bytes allocated
==1269==
==1269== 25,344 bytes in 99 blocks are definitely lost in loss record 1 of 1
==1269==   at 0x4848899: malloc (in /usr/libexec/valgrind/vgpreload_memcheck-amd64-linux.so)
==1269==   by 0x10919D: main (programa.c:14)
==1269==
==1269== LEAK SUMMARY:
==1269==   definitely lost: 25,344 bytes in 99 blocks
==1269==   indirectly lost: 0 bytes in 0 blocks
==1269==   possibly lost: 0 bytes in 0 blocks
==1269==   still reachable: 0 bytes in 0 blocks
==1269==   suppressed: 0 bytes in 0 blocks
==1269==
==1269== ERROR SUMMARY: 1 errors from 1 contexts (suppressed: 0 from 0)
```

21. Como podemos ver en este ejemplo, se pierden 99 bloques con un total de 25,344 bytes como esperábamos. El enunciado **definitely lost** quiere decir que hay **memory leak** y requiere nuestra atención. Si tuviéramos que colocar la llamada **free()** dentro del bucle, definitivamente se perderían 0 bloques de memoria. El programa nos indica que el causante es la instrucción **malloc** en la línea 14 de nuestro programa.

```
12     for (count = 0; count < LOOPS; count++)
13     {
14         pointer = (char *)malloc(sizeof(char) * MAXSIZE);
15     }
16
```

22. Enhorabuena. Ha creado su programa correctamente.